

Exploiter et payer

CloudWatch, facturation, boto3 — le quotidien du métier

CL-AWS1 — Jour 8 — Dernière journée du bloc : superviser, compter, automatiser

Objectifs de la journée

À la fin de la journée, vous saurez :

- Lire les **métriques CloudWatch** (EC2, ALB, RDS), poser des **alarmes** et construire un **dashboard**.
- Centraliser des journaux dans **CloudWatch Logs** et les interroger avec **Logs Insights**.
- Notifier un humain avec **SNS** — la chaîne complète métrique → alarme → e-mail.
- Répondre à « **qui a fait quoi ?** » avec CloudTrail.
- Maîtriser la facture : **Cost Explorer, budgets, tags de coût**, calcullette.
- Automatiser en Python avec **boto3** : inventaire, snapshots, scripts d'exploitation.
- Situer votre travail dans le **Well-Architected Framework** — la synthèse des 8 jours.

Ce soir : teardown de **fin de bloc** (tout, vraiment tout) et cap sur le **TP2**.

Plan de la journée

1. **CloudWatch métriques et alarmes** — voir, mesurer, être réveillé.
2. **Logs et SNS** — les journaux à l'échelle du compte, la notification.
3. **CloudTrail** — l'audit : qui a fait quoi, quand, d'où.
4. **La facture** — Cost Explorer, budgets, tags, calcullette.
5. **boto3** — Python retrouve AWS : lire, agir, industrialiser.
6. **Well-Architected + synthèse du bloc** — et le grand quiz des 8 jours.

Le fil du cursus, au complet aujourd'hui : **automatiser** (boto3), **superviser** (CloudWatch), **sécuriser** (CloudTrail).

1. CloudWatch — métriques et alarmes

CloudWatch : le système nerveux du compte

Depuis 8 jours, vous surveillez vos ressources en tapant **describe-....** Un admin ne regarde pas : il **instrumente**. CloudWatch collecte déjà, gratuitement, sans agent :

- des **métriques** : des séries de mesures horodatées, rangées par **namespace** (**AWS/EC2**, **AWS/ApplicationELB**, **AWS/RDS**...) et identifiées par **dimensions** (quelle instance ? quel ASG ? quelle base ?) ;
- au pas de **5 minutes** par défaut (1 min avec le *detailed monitoring*, payant) ;

- conservées jusqu'à 15 mois (en résolution décroissante).

```
aws cloudwatch list-metrics --region eu-west-3 \
  --namespace AWS/EC2 --metric-name CPUUtilization \
  --query 'Metrics[[]].Dimensions' --output table
```

Tout ce qu'on a construit cette semaine émet **déjà** des métriques. Il n'y a qu'à s'en servir.

Les métriques à connaître pour NOTRE architecture

Namespace	Métrique	Ce qu'elle raconte
AWS/EC2	CPUUtilization	saturation de calcul (le moteur du target tracking !)
AWS/EC2	StatusCheckFailed	l'instance ou son hôte est en détresse
AWS/ApplicationELB	HealthyHostCount / UnHealthyHostCount	LA métrique de dispo derrière un ALB
AWS/ApplicationELB	HTTPCode_Target_5XX_Count	l'application renvoie des erreurs
AWS/ApplicationELB	TargetResponseTime	la latence vue par l'ALB
AWS/RDS	CPUUtilization, DatabaseConnections	la base sature / fuite de connexions
AWS/RDS	FreeStorageSpace	le disque plein = base morte (à alarmer !)

Statistiques : **Average**, **Maximum**, **Sum**, percentiles... Piège classique : une instance à 100 % + une à 2 % = **Average 51 %** — parfois c'est **Maximum** qu'il faut regarder.

Ce que CloudWatch ne voit PAS (question d'examen)

Les métriques natives viennent de **l'hyperviseur** — il voit la machine de l'extérieur :

-  CPU, réseau, E/S disque, status checks ;
-  **la mémoire (RAM)** ;
-  **l'espace disque utilisé** dans le système de fichiers ;
-  les processus, votre application.

Pour voir l'intérieur : installer **l'agent CloudWatch** sur l'instance → il pousse des **métriques custom** (RAM, disque, ce que vous voulez) et des **logs**. 💰 Les métriques custom sont facturées (~0,30 \$/métrique/mois).

À réciter : « *CloudWatch ne voit pas la RAM sans agent.* » — l'examen adore, et les incidents de prod « pourquoi mon alarme mémoire n'existe pas ? » aussi.

L'alarme : cinq curseurs, pas de magie

Une **alarme** surveille UNE métrique et change d'état selon un seuil :

```
aws cloudwatch put-metric-alarm --region eu-west-3 \
  --alarm-name ${PREFIX}-cpu-asg-haute \
  --namespace AWS/EC2 --metric-name CPUUtilization \
  --dimensions "Name=AutoScalingGroupName,Value=${PREFIX}-asg-app" \
  --statistic Average \           # 1. quelle lecture de la métrique
  --period 300 \                 # 2. sur des fenêtres de combien
  --threshold 70 \               # 3. quel seuil
  --comparison-operator GreaterThanThreshold \ # 4. dans quel sens
  --evaluation-periods 2 \       # 5. combien de fenêtres consécutives
  --alarm-actions <ARN_SNS> --ok-actions <ARN_SNS>
```

`evaluation-periods 2 × period 300` = il faut **10 minutes** au-dessus de 70 % pour sonner : c'est le filtre anti-fausse-alertes. Trop réactif = astreinte qui hurle pour rien ; trop lent = incident découvert par les clients.

États d'alarme et données manquantes

Trois états, tous les trois **normaux** à connaître :

- **OK** — la condition n'est pas remplie ;
- **ALARM** — la condition est remplie → les **actions** partent (SNS, scaling...) ;
- **INSUFFICIENT_DATA** — pas assez de points pour juger : une alarme **neuve**, ou une ressource **détruite** (l'ASG du soir !).

Le réglage fin qui va avec : `--treat-missing-data`

- **notBreaching** — pas de données = tout va bien (notre choix : l'ASG est détruit chaque soir, on ne veut pas d'alerte fantôme) ;
- **breaching** — pas de données = ALARME (parfait pour « ce batch doit émettre chaque nuit » : le silence EST l'anomalie).

Les alarmes servent aussi de moteur : c'est une alarme CloudWatch qui déclenche le **target tracking** du J6 — vous en avez déjà en production sans le savoir.

Dashboards : l'état du système en un regard

Un **dashboard** CloudWatch = une grille de widgets (graphes de métriques, états d'alarmes, requêtes de logs) :

- le réflexe : **un dashboard par application** — pour StockLine : **HealthyHostCount**, **TargetResponseTime**, **HTTPCode_Target_5XX_Count**, CPU de l'ASG, **DatabaseConnections** et **FreeStorageSpace** de RDS ;
- il se **crée en JSON** (`aws cloudwatch put-dashboard`) : versionnable, reproductible — un dashboard cliqué se perd, un dashboard codé se redéploie ;
- 💰 3 dashboards gratuits (50 métriques), largement assez.

L'usage pro : le dashboard n'est pas là pour être contemplé — il sert **pendant l'incident** (où est le rouge ?) et **après** (post-mortem). Le reste du temps, ce sont les alarmes qui regardent à votre place.

2. Logs et SNS

CloudWatch Logs : journalctl à l'échelle du compte

Rappel CL-LINUX : les journaux vivent sur la machine (`journalctl`, `/var/log`). Problème depuis le J6 : **vos machines sont jetables** — leurs logs meurent avec elles. La réponse :

- **Log group** : le conteneur logique — un par application (`/stockline/api`) ; porte la **rétenion** (⚠️ défaut : conservation infinie = facture infinie ; réglez 7-30 jours) ;
- **Log stream** : le flux d'UNE source (une instance, un conteneur) dans le groupe ;
- qui écrit dedans ? L'**agent CloudWatch** installé sur l'instance (il lit `/var/log/...` ou `journald` et pousse) — c'est le rôle IAM `CloudWatchAgentServerPolicy` qu'on traîne depuis le J6 ; les services managés (RDS, Lambda...) savent aussi exporter directement.

💰 ~0,57 \$/Go ingéré, 5 Go/mois gratuits. Le log bavard en boucle est un poste de coût réel.

Logs Insights : le grep managé

Interroger des Go de logs sans SSH ni `grep` — un langage de requête simple :

```
fields @timestamp, @message
| filter @message like /ERROR/
| sort @timestamp desc
| limit 20
```

```
# Compter les erreurs par tranche de 5 minutes (la courbe d'un incident) :
filter @message like /ERROR/
| stats count(*) as erreurs by bin(5m)
```

- Portée : un ou plusieurs **log groups**, sur une fenêtre de temps ;
- 💰 facturé au **Go scanné** → toujours restreindre la fenêtre de temps ;
- l'équivalent mental : `journalctl -u stockline | grep ERROR`, mais sur les 6 instances que l'ASG a créées et détruites cette semaine — y compris les mortes.

SNS : notifier un humain (ou un système)

SNS (Simple Notification Service) = un canal de diffusion **pub/sub** :

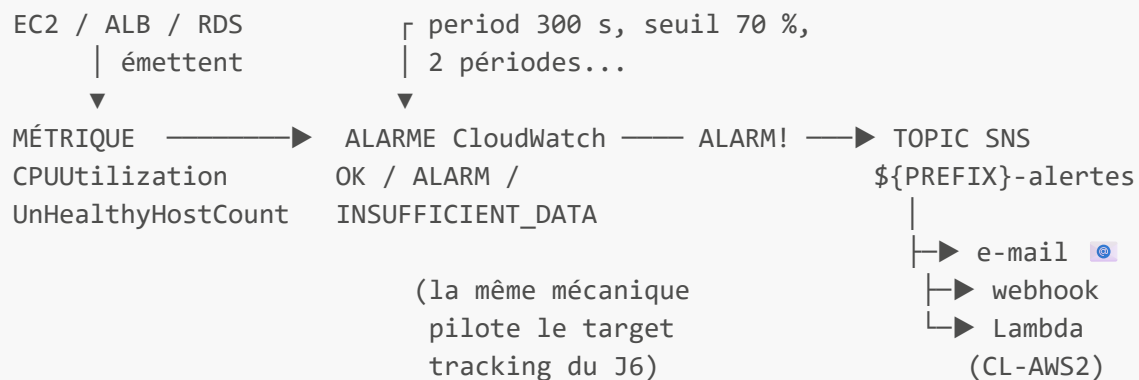
- un **topic** (`${PREFIX}-alertes`) reçoit des messages ;

- des **abonnements** les distribuent : **e-mail** (notre choix — à **confirmer** dans sa boîte, sinon rien ne part !), SMS, HTTPS/webhook (→ Slack/Teams), SQS, Lambda ;
- qui publie ? Les **alarmes CloudWatch** (`--alarm-actions`), vos scripts (`aws sns publish`, `boto3 .publish()`), d'autres services.

```
aws sns create-topic --name ${PREFIX}-alertes
aws sns subscribe --topic-arn <ARN> --protocol email \
  --notification-endpoint vous@exemple.fr
aws sns publish --topic-arn <ARN> --subject "Test" --message "Bonjour"
```

💰 1 000 e-mails/mois gratuits. On recroisera SNS au bloc CL-AWS2 dans son rôle de **découplage** entre services — aujourd'hui, c'est notre sonnette.

La chaîne complète — celle de la démo



Ce matin en démo : stress CPU réel → alarme ALARM → e-mail reçu
→ ET scale-out de l'ASG déclenché. Supervision et élasticité : le même moteur.

3. CloudTrail

Qui a fait quoi, quand, depuis où

CloudTrail : le journal d'audit du compte

Question du patron à l'exercice 8-1 : « qui a créé toutes ces machines ? ». CloudWatch ne le sait pas.

CloudTrail, si :

- il enregistre **chaque appel d'API** du compte — console, CLI, boto3, tout est appel d'API : `RunInstances`, `DeleteDBInstance`, `CreateUser`... ;
- chaque événement dit : **qui** (l'identité IAM), **quoi**, **quand**, **d'où** (IP source), **avec quel résultat** ;

- gratuit : **90 jours** d'historique consultables (*Event history*) ; au-delà, un *trail* qui archive vers S3 (bloc CL-SECU).

```
aws cloudtrail lookup-events --region eu-west-3 \
  --lookup-attributes AttributeKey=EventName,AttributeValue=TerminateInstances \
  --max-results 5 \
  --query 'Events[].[EventTime,Username,EventName]' --output table
```

Qui a tué l'instance pendant la démo du J6 ? La réponse est dedans, avec votre nom.

CloudWatch vs CloudTrail : ne les confondez plus jamais

	CloudWatch	CloudTrail
Question	« Comment ça va ? »	« Qui a fait quoi ? »
Contenu	métriques, logs, alarmes	appels d'API (audit)
Usage type	incident, performance, alerte	sécurité, forensique, conformité
Exemple	« la CPU dépasse 70 % »	« ihab a supprimé la base à 17 h 42 depuis 92.x.x.x »

Les deux se complètent : CloudWatch vous dit que la base a disparu du radar, CloudTrail vous dit **qui** l'a supprimée.

Question d'examen récurrente sous des formes infinies. La clé mnémotechnique : Watch = **regarder l'état**, Trail = **la piste** laissée par les acteurs.

En entreprise : CloudTrail activé partout, tout le temps, sans exception — c'est souvent une obligation réglementaire (on y revient au bloc CL-SECU avec GuardDuty).

4. La facture

Cost Explorer, budgets, tags — payer ce qu'on a décidé de payer

Cost Explorer : lire avant d'optimiser

Console → **Billing and Cost Management** → **Cost Explorer** : la facture en graphiques, filtrable et groupable (par **service**, par **tag**, par région, par jour).

La lecture mensuelle de l'admin (30 min qui valent cher) :

1. **Groupé par service** : quel est le top 3 ? Est-il explicable ?
2. La ligne mystère « **EC2-Other** » : c'est là que se cachent **EBS, snapshots, NAT Gateway, data transfer** — les fuites classiques ;
3. **Groupé par tag** **Projet** : qui coûte quoi (à condition d'avoir tagué...) ;

4. Toute ressource inexpliquée = un teardown raté quelque part.

Sur vos sandbox : vous devez savoir expliquer **chaque ligne** de la semaine (ALB du J6, NAT GW, RDS...). Si une ligne vous surprend, c'est qu'il reste quelque chose à détruire ce soir.

Budgets : le détecteur de fumée

AWS Budgets : un montant mensuel + des seuils d'alerte (e-mail via SNS) :

- alertes sur le **réel** (« 80 % consommés ») et sur le **prévisionnel** (« à ce rythme, vous finirez à 130 % ») — le prévisionnel prévient AVANT la casse ;
- 💰 2 budgets gratuits par compte ;
- ⚠️ un budget **alerte, n'arrête rien** : détecteur de fumée, pas extincteur. (Les actions automatiques existent — hors périmètre du bloc.)

```
aws budgets create-budget --account-id <ID_COMPTE> \
  --budget '{"BudgetName":"budget-formation","BudgetLimit":
    {"Amount":"10","Unit":"USD"},"TimeUnit":"MONTHLY","BudgetType":"COST"}' \
  --notifications-with-subscribers '[{"Notification":
{"NotificationType":"FORECASTED",
  "ComparisonOperator":"GREATER_THAN","Threshold":80},
  "Subscribers":[{"SubscriptionType":"EMAIL","Address":"vous@exemple.fr"}]]'
```

À poser sur votre sandbox **aujourd'hui** si ce n'est pas déjà fait : 10 \$, alerte à 80 %.

Les tags de coût : la comptabilité analytique du cloud

Sans tags, Cost Explorer sait dire « EC2 : 300 \$ » — mais pas **pour qui**. La stratégie minimale, trois clés sur TOUTE ressource :

- **Projet** — ventiler par produit (*stockline*, *interne*) ;
- **Environnement** — *prod* / *staging* / *dev* (et repérer le dev qui tourne la nuit) ;
- **Propriétaire** — qui prévenir avant de supprimer.

Deux étapes qu'on oublie :

1. taguer à la **création** (nos scripts le font : `--tag-specifications`, `--tags`) — le rattrapage a posteriori est une purge ;
2. **activer** les clés comme *cost allocation tags* dans Billing — sinon elles n'apparaissent jamais dans Cost Explorer.

Et pour estimer AVANT de construire : la **calcullette** — *calculator.aws* (le devis de votre architecture TP2, demandé dans la grille).

💰 La leçon de coûts du bloc, en un tableau

💰 **Ce que les 8 jours ont facturé (ou auraient pu)**

Ressource	Free-tier ?	Le piège
EC2 t3.micro	✅ 750 h/mois	plusieurs instances simultanées = dépassement
NAT Gateway	❌ ~0,05 \$/h	facture même sans trafic — 1,20 \$/jour
ALB	❌ ~0,027 \$/h + LCU	facture même sans trafic — 0,70 \$/jour
RDS db.t3.micro	✅ single-AZ	Multi-AZ ou 2^e instance = payant
Snapshots (EBS/RDS)	~	les manuels survivent aux instances
S3 / CloudFront	✅ largement	data transfer sortant à grande échelle
EIP	✅ si attachée	non attachée = facturée

Le réflexe qui résume le bloc : **ce qui se crée le matin se détruit le soir, et on le prouve.**



Exercice 8-1 — Lire une facture AWS et trouver les économies

Fichier : [exercices/06-cl-aws1/exercice-8-1-lire-la-facture.md](#) — **40 min, en binômes**

La facture de 487 \$/mois de la PME « Atelier Nordique », ligne par ligne :

- classer chaque ligne : OK / à optimiser / à supprimer — **économies chiffrées** (calculs posés !) ;
- il y a plus de 350 \$/mois à récupérer : sur-dimensionnement, staging fantôme, volumes détachés, 214 snapshots, EIP orphelines... ;
- puis les trois questions du patron : le budget qui protège, les tags qui expliquent, l'extinction nocturne (et sa subtilité RDS) ;
- partie 3 : **votre** Cost Explorer — chaque ligne de votre semaine doit s'expliquer.

Les trois fuites que vous y trouverez sont les trois fuites de tous les comptes réels.

5. boto3

Python retrouve AWS — la console, c'est fini

Session, client, resource

boto3 = le SDK AWS pour Python (`pip install boto3`). Trois objets à situer :

```
import boto3

# La Session : credentials + région. AUCUNE clé dans le code :
# boto3 reprend la config de l'AWS CLI (aws configure), ou le rôle IAM.
session = boto3.Session(region_name="eu-west-3")

# Le client : l'API brute d'un service — les mêmes verbes que la CLI.
ec2 = session.client("ec2")          # describe_instances, create_snapshot...
```



```
# La resource : une surcouche objet, plus confortable, couverture partielle.
s3 = session.resource("s3")          # s3.Bucket("x").objects.all()
```

La correspondance à retenir : `aws ec2 describe-instances` ↔ `ec2.describe_instances()` — savoir lire la doc CLI, c'est savoir écrire du boto3. Valeur sûre à long terme : le **client**.

Lire : describe + paginator (le réflexe obligatoire)

```
ec2 = boto3.client("ec2", region_name="eu-west-3")

# ❌ Naïf : ne renvoie qu'UNE page (les grands comptes débordent) :
reponse = ec2.describe_instances()

# ✅ Le réflexe pro : le paginator parcourt TOUTES les pages :
for page in ec2.get_paginator("describe_instances").paginate(
    Filters=[{"Name": "instance-state-name", "Values": ["running"]}]):
    for reservation in page["Reservations"]:
        for inst in reservation["Instances"]:
            print(inst["InstanceId"], inst["InstanceType"],
                  inst["Placement"]["AvailabilityZone"])
```

Les filtres s'exécutent **côté serveur** (comme `--filters` en CLI) : moins de données transférées, moins de code. Les réponses sont des dictionnaires : tout ce que vous savez de CL-PYTHON J2 s'applique.

Agir : snapshot, arrêt — et le garde-fou

```
# Créer un snapshot EBS (la sauvegarde scriptée du J5, en 3 lignes) :
snap = ec2.create_snapshot(
    VolumeId="vol-0abc123",
    Description="sauvegarde quotidienne stockline",
)
print(snap["SnapshotId"], snap["State"])          # snap-0... pending

# Arrêter des instances :
ec2.stop_instances(InstanceIds=["i-0abc123"])

# Attendre un état (le pendant Python de `aws ec2 wait`) :
ec2.get_waiter("snapshot_completed").wait(SnapshotIds=[snap["SnapshotId"]])
```

La règle des scripts qui **agissent** : simulation par défaut, action sur `--apply` explicite, chaque action **journalisée** (`logging`, pas `print`). Vous l'imposerez à votre propre script dans l'exercice 8-2 — et le TP2 le note.

Le script du jour : l'inventaire du compte

Fichier : `code/06-cl-aws1/inventaire-boto3.py` — à lire, exécuter, adapter :

- une `Session`, six inventaires : **EC2, volumes, snapshots, S3, RDS, load balancers** — paginators partout ;
- double sortie (réflexe CL-PYTHON J4) : tableaux pour l'humain + `inventaire.json` pour la machine ;
- et la dernière ligne qui compte : « 💰 *ATTENTION : N ressource(s) potentiellement facturée(s) encore actives* » — **votre teardown du soir devient un script.**

```
python3 inventaire-boto3.py
# ...
# 💰 ATTENTION : 4 ressource(s) potentiellement facturée(s) encore actives
```

Ce script est l'ancêtre direct du script d'exploitation demandé au TP2. Comprenez-le ligne à ligne : vous allez le prolonger.



Démo 8-1 — Alarmes, SNS et boto3 en action

Fichier : `demos/06-cl-aws1/demo-8-1-cloudwatch-boto3.md` — **60 min**

1. `alarmes-cloudwatch.sh` : topic SNS + abonnement e-mail (**confirmé en live**) + 3 alarmes (CPU ASG, hôtes malades ALB, CPU RDS) ;
2. le **stress CPU réel** sur une instance → l'alarme passe **ALARM** → **l'e-mail arrive en salle** → et le target tracking du J6 déclenche un scale-out : supervision et élasticité, même moteur ;
3. Logs Insights : la requête `filter @message like /ERROR/` ;
4. `inventaire-boto3.py` : l'état complet du compte en Python, et un snapshot créé/supprimé dans l'interpréteur.

Puis le geste final : le **teardown de fin de bloc**, vérifié... par le script d'inventaire.



Exercice 8-2 — Le gardien du compte (boto3)

Fichier : `exercices/06-cl-aws1/exercice-8-2-script-boto3-gardien.md` — **60 min, seul(e)**

Votre premier script d'exploitation complet : `gardien.py`

- **détecter** : toute instance `running` sans tag `Projet` est présumée oubliée (paginator, tags en dict, `logging` horodaté) ;
- **agir** : snapshot de ses volumes, puis `stop_instances` — jamais terminate ;
- **le garde-fou** : simulation par défaut, action uniquement avec `--apply` ;
- deux instances témoins `t3.micro` à créer... et à détruire (partie 3 : `teardown`).

Bonus : publication du rapport dans le topic SNS, et la variante « extinction nocturne » qui réalise l'économie chiffrée à l'exercice 8-1. En CL-AWS2, ce script deviendra une **Lambda planifiée**.

6. Well-Architected Framework

Les 6 piliers — la grille de relecture de tout ce que vous construirez

Les 6 piliers (1/2) — et où vous les avez DÉJÀ pratiqués

Le **Well-Architected Framework** : la grille officielle AWS pour juger une architecture. Six piliers — et vous avez passé 8 jours à les pratiquer sans le savoir :

Pilier	La question	Vous l'avez fait...
Excellence opérationnelle	Sait-on exploiter, mesurer, améliorer ?	scripts rejouables, teardown prouvé, inventaire boto3 (J8)
Sécurité	Moindre privilège, chiffrement, traçabilité ?	IAM/MFA (J1), SG chaînés (J3/J6), bucket privé + OAC (J7), CloudTrail (J8)
Fiabilité	Survit-on à la panne ?	Multi-AZ (J5), ALB + ASG min 2 (J6), sauvegardes/PITR (J5)

Les 6 piliers (2/2)

Pilier	La question	Vous l'avez fait...
Efficacité des performances	Les bonnes ressources, au bon endroit ?	classes adaptées (J2/J5), read replicas (J5), CloudFront (J7)
Optimisation des coûts	Paie-t-on ce qu'on a décidé ?	free-tier d'abord, encadrés 💰, teardown du soir, Cost Explorer/budgets/tags (J8)
Durabilité	L'empreinte est-elle minimisée ?	éteindre ce qui ne sert pas, dimensionner juste — le teardown est aussi écologique

Deux usages concrets : la **grille du TP2** est organisée sur ces piliers (votre document d'architecture devra s'y référer) ; et la **SAA-C03** structure ses domaines dessus (CL-CERT). Ce n'est pas de la théorie décorative : c'est la check-list de relecture avant chaque mise en production.

Synthèse du bloc

8 jours, une architecture complète

La carte des 8 jours

J1	compte, IAM, CLI	— l'identité et l'outillage
J2	EC2	— le calcul (AMI, user data, EBS)

- J3 VPC — le réseau (subnets, IGW/NAT, SG)
- J4 S3 + rôles IAM — les objets et les identités de machines
- J5 RDS — l'état, managé (Multi-AZ vs replicas, PITR)
- J6 ALB + ASG — la haute dispo et l'élasticité (stateless !)
- J7 Route 53, CloudFront, ACM — l'exposition au monde (DNS, CDN, HTTPS)
- J8 CloudWatch, coûts, boto3, Well-Arch — l'exploitation (alarmes, facture, boto3, les 6 piliers)

Chaque brique a été construite EN CLI, détruite chaque soir, et re-construite par script.

Le TP2 = l'assemblage, en autonomie, documenté, noté.

Cap sur le TP2 — 4 jours, notés

CL-TP2 : l'architecture 3-tiers StockLine, de bout en bout (dès demain, 4 jours) :

- **VPC multi-AZ (J3) → RDS PostgreSQL Multi-AZ (J5) → ALB + EC2 en Auto Scaling, sous-réseaux privés (J6) → front S3 + CloudFront + OAC (J7) → CloudWatch : alarmes + dashboard (J8) → scripts boto3 d'exploitation (J8) ;**
- livrables : l'infrastructure démontrée + document d'architecture (schéma, choix justifiés via les piliers, estimation de coût à la calculette) + scripts ;
- **soutenance courte + teardown noté** (prouvé, commandes vides à l'appui) ;
- la grille d'évaluation vous est remise au briefing — aucun piège : tout a été fait cette semaine, au moins une fois.

Conseil de préparation pour ce soir : relire votre journal du mini-TP et la cheatsheet du bloc. C'est tout. Dormez.

Teardown de FIN DE BLOC — le grand ménage

Aujourd'hui on ne range pas la journée : on rend un compte **vide** (checklist complète : guide formateur) :

```
# 1. Alarmes, topic SNS :
aws cloudwatch delete-alarms --region eu-west-3 --alarm-names \
  ${PREFIX}-cpu-asg-haute ${PREFIX}-alb-hotes-malades ${PREFIX}-rds-cpu-haute
aws sns delete-topic --region eu-west-3 --topic-arn <ARN_TOPIC>

# 2. ALB/ASG, VPC (+ rôle) :
./teardown-alb-asg.sh && ./teardown-vpc.sh

# 3. RDS + TOUS les snapshots manuels, buckets, log groups, EIP :
aws rds describe-db-snapshots --region eu-west-3 --snapshot-type manual \
  --query 'DBSnapshots[].DBSnapshotIdentifier'
aws s3 ls # chaque bucket restant : justifié ou détruit

# 4. Le verdict, rendu par VOTRE outil :
```

```
python3 code/06-cl-aws1/inventaire-boto3.py
# → "(aucune ressource)" partout = bloc terminé proprement
```

Récap — les points clés du jour

- **CloudWatch** : les métriques existent déjà ; une alarme = métrique + statistique + période + seuil + périodes d'évaluation ; la RAM exige l'**agent**.
- **Logs** : groupes (rétention !), flux, **Logs Insights** = grep managé ; **SNS** notifie (abonnement à confirmer).
- **CloudTrail** = qui a fait quoi ; CloudWatch = comment ça va. Jamais interchangeables.
- **Facture** : Cost Explorer chaque mois, budget avec alerte prévisionnelle, 3 tags partout, « EC2-Other » = la cachette des fuites.
- **boto3** : session → client → paginator ; scripts qui agissent = simulation par défaut + logging.
- **Well-Architected** : 6 piliers = la grille du TP2 et de la SAA.

Grand quiz de synthèse AWS1 (1/5) — les 8 jours

Question 1 — (J1) Votre binôme vous propose de travailler avec les clés du compte root « parce que ça marche à tous les coups ». Donnez deux raisons de refuser et la bonne pratique correspondante.

Question 2 — (J2/J3) Une instance EC2 lancée dans un sous-réseau privé doit télécharger des paquets apt. Par quel chemin sort-elle, quelle ressource est indispensable, et que coûte cette ressource ?

Grand quiz de synthèse AWS1 (2/5)

Question 3 — (J3/J6) Expliquez la chaîne complète des security groups de l'architecture 3-tiers (qui autorise quoi, depuis quelle source) et pourquoi aucune règle ne contient d'adresse IP.

Question 4 — (J4/J7) Quelle est la différence entre l'hébergement statique S3 public du J4 et le montage S3 privé + CloudFront + OAC du J7 ? Citez deux gains concrets du second.

Grand quiz de synthèse AWS1 (3/5)

Question 5 — (J5) Multi-AZ, read replica, snapshot : pour chacun, citez LE risque qu'il couvre — et le risque qu'aucun des trois premiers ne couvre sans le troisième.

Question 6 — (J6) Décrivez ce qui se passe, étape par étape et avec les ordres de grandeur de temps, entre l'instant où une instance d'un ASG (min 2, health check ELB) meurt et le retour à la normale.

Grand quiz de synthèse AWS1 (4/5)

Question 7 — (J7) Pour app.stockline.fr en HTTPS devant CloudFront + ALB : combien de certificats ACM, dans quelles régions, et pourquoi la validation DNS plutôt qu'e-mail ?

Question 8 — (J8) Votre alarme `UnHealthyHostCount > 0` est en `INSUFFICIENT_DATA` depuis hier soir. Donnez deux explications possibles et celle qui est la plus probable un matin de formation.

Grand quiz de synthèse AWS1 (5/5)

Question 9 — (J8) Un script boto3 d'exploitation professionnel respecte trois règles vues aujourd'hui (une sur la lecture des résultats, une sur le déclenchement des actions, une sur la trace). Lesquelles, et pourquoi chacune ?

Question 10 — (Synthèse) Reliez chaque jour du bloc (J5, J6, J7, J8) au pilier Well-Architected qu'il illustre le mieux, en justifiant d'une phrase par jour.

Réponses détaillées : guide formateur du bloc (jours 5-8).

Bravo — le bloc CL-AWS1 est terminé !

Rendez-vous demain pour le TP2

8 jours, une architecture 3-tiers complète, construite et détruite en CLI. Demain matin : briefing du **TP2 (noté, 4 jours)** — VPC, RDS Multi-AZ, ALB + ASG, S3 + CloudFront, CloudWatch, boto3. Tout ce qu'il faut est déjà dans vos mains.

🔪 Avant de partir : teardown de fin de bloc exécuté, [inventaire-boto3.py](#) rend un compte vide, budget en place.